ISSUE 日本語解説資料

Issue #47 GCP baseline design の議論たたき台を作成する

GitHub で closed になっている issue 記録を、そのまま参照用 PDF に変換したもの。

Issue #47　更新: 2026/03/12　ソース: docs/portal/issues/issue-47-gcp-baseline-design.md

GitHub: https://github.com/PLAYER1-r7/MultiCloudProject/issues/47

計画 portal architecture GCP

この ISSUE の要点

AWS-first の portal baseline は production governance まで固まったが、将来の GCP 対応をどの単位で始めるか、どこまでを AWS と共通 contract にし、どこからを GCP 固有 implementation として扱うかはまだ未整理である。このままだと、GCP 対応を始める際に static hosting、custom domain、DNS ownership、CI/CD、security baseline、observability、rollback の論点が一度に混ざり、Issue 8 で固定した portability

目標

GCP baseline design の議論を始めるためのたたき台を作り、target architecture、delivery path、security/operations baseline、非対象、open questions を small-scope で切り出せる状態にする。

完了状態

ISSUE CLOSED

boundary を崩しやすい。

1. この Issue が扱っていたこと

まず Summary と Goal から、この Issue が何を決めるためのものだったかを短く押さえる。

AWS-first の portal baseline は production governance まで固まったが、将来の GCP 対応をどの単位で始めるか、どこまでを AWS と共通 contract にし、どこからを GCP 固有 implementation として扱うかはまだ未整理である。このままだと、GCP 対応を始める際に static hosting、custom domain、DNS ownership、CI/CD、security baseline、observability、rollback の論点が一度に混ざり、Issue 8 で固定した portability boundary を崩しやすい。

目標

GCP baseline design の議論を始めるためのたたき台を作り、target architecture、delivery path、security/operations baseline、非対象、open questions を small-scope で切り出せる状態にする。

2. 完了判断の見方

Tasks、Definition of Done、Current Status の記述から、どこまで完了済みとして扱っているかを先に読む。

TASKS

4/4 項目が完了済みとして記録されている。

DEFINITION OF DONE

4/4 項目が完了済みとして記録されている。

ISSUE CLOSED / GitHub Issue: #47 / GitHub URL: https://github.com/PLAYER1-r7/MultiCloudProject/issues/47 / Sync Status: local close note updated and GitHub issue body resynced before closing / Close Status: GitHub issue closed; local record retained as final reference / local issue record として GCP baseline design の議論たたき台を作成した / preview domain の第一候補は `preview.gcp.ashnova.jp` とした / GCP の第一候補 architecture は `Cloud Storage + global external HTTPS load balancer + Cloud CDN + Cloud Armor + Google-managed certificate` とした / open questions は Resolution へ統合し、GCP first-step scope と provisional direction は固定した / follow-up chain は Issue 48 から Issue 56 まで起票済みであり、Issue 48 から Issue 56 までの Resolution は固定済みである / follow-up chain のうち Issue 52 / 53 の implementation と validation は完了して CLOSED であり、本 issue は GCP preview planning chain の上位基準として参照する状態である / GitHub Issue #47 は local issue record の最新版へ再同期済みである

3. 依存関係と読み進め方

Dependencies や前提 Issue がある場合は、ここで関連先を先に把握しておくと読みやすい。

依存関係

- Issue 8
- Issue 9
- Issue 10
- Issue 11
- Issue 12
- Issue 14
- Issue 46

読み方の注意

この解説書は原文の意思決定や完了状態を読みやすくするための補助資料である。最終的な正本は原文 Issue にあり、判断変更時は原文側を先に更新する。

4. 用語集

原文に残る英語や実務用語のうち、訳し切らずに補助説明を付けた方が読みやすい語をここにまとめる。

原語	日本語の扱い	補足
baseline	基準線	後続作業が参照する最小限の判断・実装・検証の出発点。
staging	ステージング	本番前に動作や反映を確認するための検証環境。
production	本番環境	実際の利用者に公開される運用環境。
operator	運用担当者	監視、承認、非表示、切り戻しなどの運用行為を担う役割。
workflow	ワークフロー	GitHub Actions などで定義された自動処理の流れ。
owner	責任者	意思決定または更新責任を持つ担当者。文脈により product owner などを含む。
review	レビュー	内容や実装が要件を満たすか確認する工程。

原語	日本語の扱い	補足
evidence	証跡	判断や完了状態の根拠になるログ、表、文書、出力結果。
draft	下書き	まだ確定前だが議論やレビューの基礎として置かれた文書。
rollback	ロールバック	変更後の状態から以前の安定状態へ戻すこと。
current status	現在の状態	Issue 文書時点での完了度合いや同期状況を示す節。
definition of done	完了条件	その Issue を完了とみなすために満たすべき条件。
task contract	タスク契約	対象、目的、スコープ、検証計画、リスクを明文化した作業契約。

5. 原文

以下に Issue 原文をそのまま掲載する。詳細な根拠、証跡、チェックリストはこの原文を参照する。

概要

AWS-first の portal baseline は production governance まで固まったが、将来の GCP 対応をどの単位で始めるか、どこまでを AWS と共通 contract にし、どこからを GCP 固有 implementation として扱うかはまだ未整理である。このままだと、GCP 対応を始める際に static hosting、custom domain、DNS ownership、CI/CD、security baseline、observability、rollback の論点が一度に混ざり、Issue 8 で固定した portability boundary を崩しやすい。

目標

GCP baseline design の議論を始めるためのたたき台を作り、target architecture、delivery path、security/operations baseline、非対象、open questions を small-scope で切り出せる状態にする。

タスク契約

タスク契約

メタデータ
- タスク ID: ISSUE-47
- タイトル: GCP baseline design の議論たたき台を作成する
- 依頼者: repository owner
- 担当者: AI agent
- 対象アプリ: portal-web
- 対象環境: future GCP planning
- 優先度: 中
- 先行条件: Issue 8 closed, Issue 9 closed, Issue 10 closed, Issue 11 closed, Issue 12 closed, Issue 14 closed, Issue 46 closed

目的
- 解決する問題: GCP 対応を始める際の最初の論点が未整理のままだと、AWS-first で確定した product/app contract と GCP 固有 implementation の境界が曖昧になり、architecture、IaC、workflow、DNS、security、operations を一度に広げてしまいやすい
- 期待する価値: GCP 対応の第一段を design baseline として切り出し、何を今回決めるか、何を後続実装 issue に回すか、どの論点から合意を始めるかを明文化できる

スコープ
- 含むもの: GCP support の意味の定義、static-first な target architecture の候補、external DNS との関係、CI/CD と IaC の候補境界、security/observability/rollback/cost の論点整理、open questions table の作成
- 含まないもの: GCP resource 作成、deploy workflow 実装、GitHub secrets / variables の実登録、Cloud DNS authority 移行、runtime multi-cloud 同時運用、AWS production path の再設計
- 編集可能パス: docs/portal/issues/issue-47-gcp-baseline-design.md
- 制限パス: apps/portal-web/**, infra/**, .github/workflows/*.yml, closed issue records except explicit evidence references

受け入れ条件
- [x] 条件 1: GCP 対応で最初に固定したい architecture / delivery / operations の論点が文書から一意に読める
- [x] 条件 2: AWS-first baseline と衝突しない scope boundary と非対象が明示されている
- [x] 条件 3: 次に起こすべき実装系 follow-up issue を切り出せる粒度で open questions と暫定方向が整理されている

実装計画
- 変更見込みファイル: docs/portal/issues/issue-47-gcp-baseline-design.md
- アプローチ: Issue 8 の portability boundary、Issue 9 の IaC policy、Issue 10 の CI/CD policy、Issue 11 の current DNS governance を前提に、GCP path を static hosting、delivery edge、DNS contract、workflow contract、security baseline、observability baseline の単位で切り分ける
- 採用しなかった代替案と理由: いきなり GCP 実装へ進む案は論点が混ざりやすく、AWS 側で確定した boundary を壊すため採らない。逆に runtime multi-cloud parity まで同時に設計する案も first step として重すぎるため採らない

検証計画

```
- 実行するテスト: markdown review; get_errors on edited files
- 確認するログ/メトリクス: scope boundary、open question coverage、GCP candidate architecture wording の整合
- 失敗時の切り分け経路: docs/portal/03_PRODUCT_DEFINITION_DRAFT.md、docs/portal/10_MULTICLOUD_CONSTRAINTS_DRAFT.md、docs/portal/11_IAC_POLICY_DRAFT.md、docs/portal/issues/issue-08-multicloud-design-constraints.md、docs/portal/issues/issue-09-iac-policy.md、docs/portal/issues/issue-10-cicd-policy.md、docs/portal/issues/issue-46-external-dns-automation-route53-judgment.md を照合し、contract drift と implementation overreach のどちらが起きているかを切り分ける

リスクとロールバック
- 主なリスク: GCP baseline design の議論が custom domain migration、full CI/CD implementation、provider-specific product selection を一度に抱え込み、planning issue なのに implementation 前提へ寄りすぎること
- 影響範囲: future GCP architecture、IaC 方針、workflow naming、DNS governance、security baseline
- 緩和策: 今回の issue では candidate baseline と open questions に留め、resource creation、workflow 実装、credential 登録、DNS authority 変更は follow-up に分離する
- ロールバック手順: scope が広がりすぎた場合は target architecture の第一候補、DNS boundary、workflow boundary、security baseline の論点だけを残し、provider product comparison や implementation detail は別 issue に切り出す
```

作業項目

- [x] GCP support の first-step scope を固定する
- [x] GCP static delivery architecture の候補を整理する
- [x] DNS / certificate / domain ownership の boundary を整理する
- [x] CI/CD / IaC / security / observability / rollback / cost の open questions を整理する

完了条件

- [x] GCP 対応の今回スコープと非対象が同じ文脈で読める
- [x] architecture、DNS、workflow、security、operations の最初の論点が過不足なく並んでいる
- [x] AWS-first baseline と衝突しない provisional direction が置かれている
- [x] follow-up issue へ分割しやすい open questions table が用意されている

Initial Notes

- Issue 8 で確定した portability boundary により、user-facing route、frontend configuration contract、frontend architecture、monitoring wording は cloud-neutral に保つ前提である
- Issue 9 により、IaC は OpenTofu を基準とし、environment separation、explicit outputs、cloud-neutral app contract を維持する前提である
- Issue 10 により、CI/CD は GitHub Actions を基準とし、validation、artifact、deploy を分離する前提である
- Issue 46 により、current production domain は external DNS を source-of-truth とし、authoritative DNS write automation は current phase で採らない前提である
- 現在の portal は static-first、no end-user login、no custom API、no application persistence を baseline とするため、GCP の第一段も public static delivery path から始めるのが自然である

Issue 47 Discussion Draft

このセクションは、GCP 対応を始めるための初期たたき台である。ここで決めたいのは「今すぐ AWS と GCP を同時本番運用する方法」ではなく、「portal を GCP へ広げる場合に、最初の一歩をどの architecture / workflow / operations boundary で始めるか」である。

1. 今回の前提整理

- portal の product/app model はすでに cloud-neutral を前提としており、GCP 対応で user-facing route や frontend architecture を作り直す必要はない
- current production governance は AWS-first であり、external DNS source-of-truth も current phase では維持する
- GCP 対応の第一段は、AWS production replacement ではなく、future expansion baseline の整理として扱う方が安全である
- したがって今回の主論点は、resource parity ではなく baseline design と scope slicing である

2. まず固定したい GCP support の意味

提案:

- GCP support の第一段は `portal-web` を `GCP` でも `static delivery` できる設計を定義すること とする
- 初回の GCP scope は public static hosting、custom-domain readiness、deploy path、security baseline、minimum observability に限定する
- dynamic backend、end-user auth、cross-cloud failover、same-hostname dual-active は含めない

この切り分けを採る理由:

- 現行 product model が static-first であり、最小の portability proof として妥当である
- GCP の最初の設計判断は hosting / edge / DNS / certificate / deploy path に集中した方が後続 issue を分けやすい
- runtime multi-cloud parity や active-active を先に持ち込むと small-team phase の現実と乖離しやすい

3. GCP target architecture の第一案

第一候補:

- Cloud Storage bucket を static asset origin とする
- Global external HTTPS load balancer を public entrypoint とする
- Cloud CDN を edge cache とする
- Cloud Armor を WAF / edge protection baseline とする
- preview domain 向け certificate は Google-managed certificate を第一候補とし、Certificate Manager は follow-up comparison に残す

この第一候補を推す理由:

- 現行 AWS path の `object storage + edge + certificate + WAF` という責務分離に近く、Issue 8 の boundary を保ちやすい
- frontend は provider-specific SDK を持たずに済み、delivery layer の差分として閉じ込めやすい
- 将来 custom domain と security posture を production-grade に寄せる場合も、Cloud Armor と managed certificate を同じ設計文脈で扱いやすい

現時点で保留する代替案:

- Firebase Hosting を第一候補にする案
  - simpler ではあるが、current OpenTofu / infra boundary と別系統の運用判断が増えやすい

- Cloud Run を static site host にする案
  - static-first の現状では compute を増やす合理性が薄い
- Cloud DNS まで first step で含める案
  - Issue 46 の external DNS source-of-truth judgment と衝突しやすく、scope が広がりすぎる

現時点の第一案:

- GCP baseline の第一候補 architecture は `Cloud Storage + global external HTTPS load balancer + Cloud CDN + Cloud Armor + Google-managed certificate` とする
- この構成は preview/static delivery proof に必要な責務を最小構成で揃えつつ、Issue 8 の cloud-neutral frontend contract を維持しやすい
- Certificate Manager や別系統 hosting product の比較は、preview path を先に成立させた後の follow-up issue へ分離する

4. DNS / custom domain / certificate のたたき台

提案:

- current phase の authoritative DNS は external DNS に残し、GCP path でも DNS write は operator-managed のままにする
- GCP 側は `reviewed target value` を出力する側 として設計し、actual DNS update は follow-up operator step に分離する
- custom domain の初回検証は AWS production と同じ hostname を奪い合わず、GCP 専用の preview domain か subdomain を前提にした方が安全である

候補例:

- `gcp.ashnova.jp`
- `www.gcp.ashnova.jp`
- `portal-gcp.ashnova.jp`
- `preview.gcp.ashnova.jp`

現時点の第一案:

- same hostname cutover は後続 issue に分離し、GCP baseline では provider-specific preview domain を前提にする
- preview domain は `preview.gcp.ashnova.jp` を第一候補とする

この preview domain を第一候補にする理由:

- `preview` を含めることで non-production path であることが domain から一意に読める
- `www.aws.ashnova.jp` と hostname が競合せず、current production monitoring / rollback / alert routing wording を壊さない
- `gcp` を 2nd-level label 側に残すため、将来 `preview.aws.ashnova.jp` や `preview.azure.ashnova.jp` のような比較軸にも拡張しやすい

5. IaC / state / output contract のたたき台

提案:

- GCP path でも IaC 標準は OpenTofu を維持する
- environment と cloud path を分けた entrypoint を採り、GCP resource definition を AWS production/staging と混ぜない
- frontend へ渡す contract は既存方針どおり provider-neutral naming を維持する

ここで最初に確認したいこと:

- remote state backend を GCP 用に GCS へ分けるか、repo 全体で現行 backend 系を維持するか
- GCP output と frontend/public contract の間にどの変換層を置くか

現時点の第一案:

- app-facing contract は neutral naming のまま維持し、GCP specific output は deploy layer で変換する
- state backend 方針は separate question として切り出し、resource topology と同一 issue に混ぜない

6. CI/CD / deploy path のたたき台

提案:

- build artifact は AWS / GCP で共通化し、deploy workflow だけを cloud-specific に分ける
- validation signal は existing portal-web build / smoke rule を流用できるかを最初に確認する
- GCP deploy workflow は first step では staging 相当か preview 相当までに留め、production promotion gate は後続 issue に分離する

この方向の理由:

- Issue 10 の `validation` と `deploy` を分離する という既存判断を保ちやすい
- GCP path の初期実装で build pipeline まで分岐させると保守対象が増えすぎる
- first GCP rollout を preview/staging 相当で止めれば、DNS・approval・incident blast radius を抑えられる

7. Security / observability / rollback のたたき台

提案:

- security baseline は HTTPS、security headers、Cloud Armor、Cloud Audit Logs、Secret Manager rotation path を最低論点とする
- observability baseline は `/`、`/overview`、`/guidance` の reachability、static asset load success、deploy evidence path を主語にする
- rollback baseline は `last known-good static artifact restore` と `DNS/operator step rollback` を分離して扱う

補足:

- monitoring wording は provider metric 名ではなく user-facing availability を主語にする
- GCP 導入でも current repo の alert routing 思想を壊さず、owner と first-response path を別 issue で定義できるようにしておく

8. Cost と rollout sequencing のたたき台

提案:

- first GCP step は production equivalence ではなく `architecture proof + preview delivery proof` を成功条件に置く
- monthly cost ceiling は AWS current ceiling をそのまま流用するのではなく、GCP proof-of-concept 用 ceiling を別途置くかを確認する
- rollout sequence は `design baseline -> preview IaC -> preview deploy -> preview smoke -> domain/certificate decision -> production judgment` の順が妥当である

9. 後続 issue へ分けたい単位

- GCP architecture baseline

- GCP IaC topology と state backend judgment
- GCP preview delivery resource execution
- GCP deploy workflow baseline
- GCP preview workflow execution
- GCP preview domain / certificate / DNS operator memo
- GCP security baseline
- GCP monitoring / alert routing baseline
- GCP rollback and recovery memo

10. この場で確認したい論点

Resolution を書く段階では、この表の `Resolution 確定文言` 列を確定文言へ更新してから使うこと。

論点	判断方向（Discussion 時点の仮）	Resolution 確定文言
GCP support の第一段を preview/static delivery baseline に限定してよいか	yes。first step は production parity ではなく static delivery proof に留める	`GCP support の第一段は preview/static delivery baseline に限定し、production parity や active-active design は後続 issue に分離する`
GCP の第一候補 architecture を何に置くか	Cloud Storage + global external HTTPS load balancer + Cloud CDN + Cloud Armor を第一候補にする	`GCP の第一候補 architecture は Cloud Storage を origin、global external HTTPS load balancer を public entrypoint、Cloud CDN を edge cache、Cloud Armor を WAF baseline とし、preview certificate は Google-managed certificate を第一候補とする`
authoritative DNS を GCP 側へ移すか	no。current phase は external DNS source-of-truth を維持する	`GCP path でも authoritative DNS は current phase では external DNS に残し、GCP 側は reviewed target value を出力する側に留める`
初回 preview domain を何に置くか	`preview.gcp.ashnova.jp` を第一候補にする	`初回の GCP preview domain は preview.gcp.ashnova.jp を第一候補とし、AWS production hostname とは分離した dedicated preview subdomain を使う`
初回 domain 検証で AWS production と同じ hostname を使うか	no。preview 専用 subdomain を使う方向が安全	`初回の GCP domain 検証は AWS production と同じ hostname を使わず、preview 専用 subdomain を前提にする`

論点	判断方向（Discussion 時点の仮）	Resolution 確定文言
IaC backend をどう分けるか	separate question として後続判断に切り出す	`GCP IaC backend の最終方針は resource topology と切り分けた後続 issue で判断し、本 issue では OpenTofu を標準とすることだけを固定する`
build/deploy workflow をどう分けるか	build artifact は共有し、deploy workflow は cloud-specific に分ける	`portal-web build artifact は共通化し、deploy workflow は cloud-specific に分離する`
security/observability の主語を何に置くか	provider metric ではなく user-facing availability に置く	`GCP 側の security/observability baseline でも、主語は provider resource 名ではなく user-facing availability と reviewable evidence path に置く`
first GCP step の成功条件を何に置くか	architecture proof と preview delivery proof に置く	`first GCP step の成功条件は production 同等性ではなく、architecture proof と preview delivery proof の成立とする`

Resolution

Issue 47 の判断結果は次の通りとする。

- GCP support の第一段は preview/static delivery baseline に限定し、production parity や active-active design は後続 issue に分離する
- GCP の第一候補 architecture は Cloud Storage を origin、global external HTTPS load balancer を public entrypoint、Cloud CDN を edge cache、Cloud Armor を WAF baseline とし、preview certificate は Google-managed certificate を第一候補とする
- GCP path でも authoritative DNS は current phase では external DNS に残し、GCP 側は reviewed target value を出力する側に留める
- 初回の GCP preview domain は `preview.gcp.ashnova.jp` を第一候補とし、AWS production hostname とは分離した dedicated preview subdomain を使う
- 初回の GCP domain 検証は AWS production と同じ hostname を使わず、preview 専用 subdomain を前提にする
- GCP IaC backend の最終方針は resource topology と切り分けた後続 issue で判断し、本 issue では OpenTofu を標準とすることだけを固定する
- `portal-web` build artifact は共通化し、deploy workflow は cloud-specific に分離する
- GCP 側の security / observability baseline でも、主語は provider resource 名ではなく user-facing availability と reviewable evidence path に置く
- first GCP step の成功条件は production 同等性ではなく、architecture proof と preview delivery proof の成立とする

この合意で明確になること:

- GCP 対応の最初のスコープは preview delivery proof に限定され、AWS production replacement や multi-cloud parity を前提にしない

- first-step architecture は static hosting と edge delivery に集中し、frontend contract や user-facing route の再設計を不要にできる
- external DNS source-of-truth judgment を維持したまま、GCP 側は target value の出力と preview validation に責務を限定できる
- build と validation の既存投資を流用しつつ、deploy workflow だけを cloud-specific に分割する方向で後続実装を切り出せる
- security、observability、rollback は provider product 名中心ではなく、user-facing availability と reviewable evidence を軸に揃えられる

後続 issue への引き継ぎ事項:

- GCP architecture baseline issue では Cloud Storage、load balancer、Cloud CDN、Cloud Armor、certificate wiring の resource topology を具体化する
- GCP IaC topology issue では backend を GCS に分けるかどうかを含む state backend judgment を扱う
- GCP preview delivery resource execution issue では `infra/modules/portal-gcp-static-delivery/`、`infra/environments/gcp-preview/`、usage-oriented output surface、blocked pending state の扱いを execution record として具体化する
- GCP deploy workflow issue では shared build artifact を前提に preview deploy path と smoke verification を定義する
- GCP preview workflow execution issue では `resource_execution_reference`、artifact provenance check、preview deploy evidence、operator handoff artifact を実装単位として具体化する
- GCP preview domain / certificate / DNS operator memo issue では `preview.gcp.ashnova.jp` を前提に reviewed target、certificate validation、operator step を整理する
- GCP security baseline issue では Cloud Audit Logs、Secret Manager rotation path、edge protection baseline、preview credential boundary を current repo の基準に接続する
- GCP monitoring / alert routing baseline issue では user-facing availability、first-response path、alert routing、operator hold 条件を current repo の基準に接続する
- GCP rollback and recovery memo issue では artifact restore、resource correction、DNS rollback boundary、post-recovery verification を current repo の基準に接続する

プロセスレビュー記録

- Section 10 の open questions は `Resolution 確定文言` 列を埋めたうえで本 Resolution に統合した
- この記録は implementation plan ではなく design baseline の確定記録であり、resource creation、workflow 実装、credential 登録、DNS authority 変更は依然として後続 issue の対象である
- GitHub Issue #47 の body は local issue record の最新版へ再同期する前提で扱う

現在の状態

- ISSUE CLOSED
- GitHub Issue: #47
- GitHub URL: https://github.com/PLAYER1-r7/MultiCloudProject/issues/47
- Sync Status: local close note updated and GitHub issue body resynced before closing
- Close Status: GitHub issue closed; local record retained as final reference
- local issue record として GCP baseline design の議論たたき台を作成した
- preview domain の第一候補は `preview.gcp.ashnova.jp` とした

- GCP の第一候補 architecture は `Cloud Storage + global external HTTPS load balancer + Cloud CDN + Cloud Armor + Google-managed certificate` とした
- open questions は Resolution へ統合し、GCP first-step scope と provisional direction は固定した
- follow-up chain は Issue 48 から Issue 56 まで起票済みであり、Issue 48 から Issue 56 までの Resolution は固定済みである
- follow-up chain のうち Issue 52 / 53 の implementation と validation は完了して CLOSED であり、本 issue は GCP preview planning chain の上位基準として参照する状態である
- GitHub Issue #47 は local issue record の最新版へ再同期済みである

依存関係

- Issue 8
- Issue 9
- Issue 10
- Issue 11
- Issue 12
- Issue 14
- Issue 46